

CHAPTER 6

IMPLEMENTATION

6.1 METHODOLOGY

The DepHNN utilized EEG data from 45 participants aged 18–25, comprising 24 healthy controls and 21 individuals clinically diagnosed with depression. Participants were screened to exclude any history of trauma or psychoactive medication use. EEG signals were recorded using a 64-channel 10–20 system at a sampling rate of 500 Hz. Ethical clearance and informed consent were obtained prior to data collection. Preprocessing included artifact removal via Independent Component Analysis (ICA), bandpass filtering (0.5–50 Hz), normalization, and segmentation into overlapping time windows to standardize input and mitigate overfitting.

A hybrid CNN-LSTM model was developed to capture both spatial and temporal features of the EEG signals. The CNN layers, comprising 64 filters with ReLU activation, extracted spatial patterns across electrodes, while two stacked LSTM layers (64 and 32 units) learned temporal dependencies across the segmented windows[6]. Two fully connected dense layers with 16 units each refined feature abstraction, and a softmax output layer performed binary depression classification. The model was trained using the Adam optimizer on stratified training (70%), validation (20%), and test (10%) splits, achieving an accuracy of **98.27%**, with precision, recall, and F1-scores exceeding 99%.

The integration of CNN and LSTM allowed simultaneous modeling of spatial topographies and temporal dynamics, while sliding window segmentation enhanced generalization and effectively increased the dataset size. The lightweight architecture ensured rapid convergence and suitability for real-time applications[15]. Prediction results were presented through a user-friendly interface displaying class labels, probability scores, and graphical EEG summaries, facilitating interpretation for clinical, telehealth, and mental wellness contexts. The architecture is also adaptable for detecting depression severity or other neuropsychiatric conditions and can be deployed on edge devices or IoMT platforms for continuous monitoring and early intervention.

6.2 ALGORITHMS

6.2.1 CONVOLUTIONAL NEURAL NETWORK (CNN)

1. Introduction and Motivation

EEG data, such as the pre-processed windows in our Model with shape **(19 channels × 256 time steps)**, is inherently high-dimensional and noisy. Detecting depression patterns from raw EEG is challenging because relevant information arises from spatial patterns across multiple channels rather than individual electrodes—for example, asymmetries between left and right frontal lobes or coordinated variations across frontal and parietal regions. Furthermore, EEG signals are often contaminated by artifacts from eye blinks, muscle activity, or external electrical sources.

The CNN serves as an automated spatial feature extractor, learning inter-channel patterns directly from the data without manual feature engineering. Rather than defining specific features like “frontal alpha asymmetry,” the CNN identifies the most predictive spatial patterns autonomously.

2. CNN Algorithm

The CNN forms the **spatial feature extraction block** of the hybrid CNN-LSTM model. Key characteristics include:

- **Input Layer:** Receives EEG windows of shape **(19 × 256)**.
- **Conv1D Layer:**
 - **Filters:** 64 filters with a kernel size of 5.
 - **Sliding Window:** Each filter slides along the time dimension, simultaneously examining all 19 channels.
 - **Spatial Feature Learning:** Filters capture diverse spatial patterns, including low/high-frequency waves, eye-blink artifacts, and frontal alpha asymmetry.
- **Activation (ReLU):** Rectified Linear Unit sets negative values to zero, preserving key activations and reducing noise.
- **Output Feature Map:** Produces a high-level spatial representation, e.g., **(64 filters × 252 time steps)**, which is then passed to LSTM layers for temporal modeling.

3. CNN Implementation Snippet:

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv1D, ReLU

model = Sequential()

# Input: (256 time steps, 19 channels)

model.add(Conv1D(filters=64, kernel_size=5, input_shape=(256, 19)))

model.add(ReLU())

# Output: feature map of shape (252, 64)
```

4. Role and Advantages of CNN

- **Automated Feature Learning:** Eliminates the need for manual spatial feature engineering.
- **Noise Reduction:** Filters emphasize meaningful signals while suppressing artifacts.
- **Dimensionality Reduction:** Converts raw EEG into a compact, structured feature map.
- **Preparation for LSTM:** Provides a spatially encoded representation for temporal modeling.

The CNN transforms raw, noisy EEG windows into high-level spatial feature maps, enabling the hybrid CNN-LSTM model to accurately predict depression by capturing both spatial and temporal brain dynamics.

6.2.2 LONG SHORT-TERM MEMORY (LSTM) NETWORK

1. Introduction and Motivation

EEG signals are inherently temporal, with depression-related patterns often emerging gradually across consecutive epochs. LSTMs, a type of Recurrent Neural Network (RNN), are designed to learn long-term dependencies in sequential data. Using internal gating mechanisms, LSTMs retain important past information while ignoring irrelevant signals, making them ideal for EEG-based depression detection. By leveraging this temporal learning capability, the model becomes more sensitive to subtle changes in neural activity that distinguish depressed individuals from healthy controls.

2. LSTM Algorithm

In the CNN-LSTM hybrid model, LSTM layers process the CNN feature map (64 filters $\times$ 252 time steps). Key components include:

- **Input Layer:** Receives CNN feature map as a sequence over time.
- **Stacked LSTM Layers:**
 - First Layer: 64 units capturing short- and mid-term temporal dependencies.
 - Second Layer: 32 units for higher-level temporal abstraction.
- **Gating Mechanisms:** Input, forget, and output gates control memory updates and flow of information.
- **Output:** Final hidden state encodes temporal EEG patterns, passed to dense layers for classification.

3. LSTM Implementation Snippet:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

model = Sequential()
model.add(LSTM(64, return_sequences=True, input_shape=(252,64)))
model.add(LSTM(32))
model.add(Dense(16, activation='relu'))
model.add(Dense(2, activation='softmax')) # Depressed / Healthy
```

4. Role and Advantages LSTM

- **Temporal Pattern Learning:** Captures evolution of EEG features over time.
- **Memory Retention:** Retains relevant past information and ignores noise.
- **Integration with CNN:** Combines spatial and temporal features for high-accuracy classification.
- **Sequence Abstraction:** Stacked layers condense temporal dynamics for final prediction.

LSTM layers complement CNN spatial extraction by modeling EEG temporal dynamics. This hybrid approach improves reliability and accuracy in depression detection by capturing both when and where brain activity patterns occur.

6.3MODULES

6.3.1 Data Acquisition and Pre-processing Module

6.3.2 Feature Extraction Module (CNN)

6.3.3 Temporal Pattern Learning Module (LSTM)

6.3.4 Rectified Linear Unit (ReLU) Activation

6.3.5 Adam Optimization

6.3.6 Huber Loss Function

6.3.7 Classification and Prediction Module

6.3.1 DATA ACQUISITION AND PREPROCESSING MODULE

The data used to train our model comes from the publicly-available EEG dataset “EEG_Data_New” (Figshare ID 4244171), which includes recordings from individuals with Major Depressive Disorder (MDD) and healthy controls. According to the dataset metadata, it contains EEG files labelled ‘H’ for healthy controls and ‘MDD’ for depression cases, under conditions such as eyes-closed (EC), eyes-open (EO) and P300 tasks. [Figshare](#)

During acquisition, EEG signals were collected across multiple channels according to standard electrode placement, and recorded under controlled conditions to reduce noise and interference. Because raw EEG signals are often contaminated by artifacts (e.g., eye blinks, muscle movements) and environmental noise, a robust preprocessing pipeline is applied. First, band-pass filtering is used to isolate relevant frequency bands associated with cognitive and emotional states. Next, Independent Component Analysis (ICA) is employed to identify and remove artifact components. Then normalization (such as z-score scaling) is applied across channels to ensure consistent magnitude and reduce subject-specific bias.

Finally, the cleaned EEG data is segmented into fixed-length windows (for example, each window capturing a defined span of time) to create structured input samples for the CNN-LSTM pipeline. Each segment is labelled according to the participant’s group (MDD vs. Healthy) from the dataset. This module therefore ensures that only clean, standardized, and labelled EEG segments from the EEG_Data_New dataset are used for feature extraction and model training, improving downstream reliability and classification performance. The segmentation process also preserves the temporal characteristics of the brain signals, which is essential for detecting depression-related neural patterns. Additionally, the module ensures uniform sample size across all recordings, allowing the deep learning architecture to process

each segment consistently. As a result, this step plays a crucial role in converting raw EEG recordings into machine-learning-ready data that supports robust model generalization.

6.3.2 FEATURE EXTRACTION MODULE (CNN)

The Convolutional Neural Network (CNN) is used to automatically extract meaningful spatial features from the preprocessed EEG segments obtained from the **EEG_Data_New** dataset. Each EEG segment contains multiple channels representing different regions of the brain, and the signals are high-dimensional with complex inter-channel dependencies. The CNN is specifically designed to detect these spatial relationships, which are critical for distinguishing between depressed and healthy neural patterns.

In the CNN, each convolutional filter slides across the temporal dimension of the EEG window, analyzing signals from all channels simultaneously. This enables the model to detect patterns such as **frontal alpha asymmetry**, hemispheric imbalances, or frequency-specific oscillations that correlate with depressive states. Following convolution, the **ReLU (Rectified Linear Unit) activation** function introduces non-linearity by setting negative activations to zero and retaining positive values, allowing the network to focus on meaningful features while suppressing noise.

Additionally, the CNN reduces dimensionality through techniques such as **weight sharing** and optional **max pooling**, which preserves essential spatial features while minimizing computational complexity. The output of this module is a set of compact, high-level feature maps that encode the spatial patterns in the EEG data. These enriched representations are then passed to the **LSTM module** for temporal pattern learning, forming a complete spatial-temporal feature extraction pipeline suitable for accurate depression detection.

6.3.3 SEQUENCE LEARNING MODULE (LSTM)

The **Long Short-Term Memory (LSTM) module** is employed to capture temporal patterns and sequential dependencies in the EEG signals. While the CNN extracts spatial features across electrode channels, the LSTM analyzes how these features evolve over time, which is crucial because depressive neural patterns often manifest as dynamic changes in brain activity rather than static snapshots. Each preprocessed EEG segment, transformed into

high-level feature maps by the CNN, is fed into the LSTM to model these temporal relationships.

The LSTM in this DepHNN consists of **two stacked layers** with 64 and 32 hidden units, respectively. Using **return sequences**, the network preserves the full temporal structure of the feature maps for downstream processing. This configuration allows the model to learn long-range dependencies across consecutive EEG windows, such as fluctuations in frontal alpha asymmetry or variations in beta rhythm patterns, which are indicative of depression. The LSTM's memory cells retain important patterns while ignoring irrelevant noise, enabling robust modeling of the sequential nature of EEG signals.

Finally, the LSTM output provides a temporally-aware feature representation that captures both the magnitude and evolution of depression-related neural activity. These enriched sequential features are then passed to the **fully connected classification module**, which consolidates spatial and temporal information to generate accurate predictions of a subject's mental state (Depressed vs. Healthy). By combining CNN and LSTM, the system achieves comprehensive **spatial-temporal modeling**, maximizing the predictive power of the EEG_Data_New dataset.

6.3.4 RECTIFIED LINEAR UNIT (RELU) ACTIVATION

The **Rectified Linear Unit (ReLU)** activation function is applied throughout the CNN–LSTM network to improve learning efficiency, model stability, and overall performance. ReLU introduces non-linearity by converting all negative input values to zero while leaving positive values unchanged. In the context of EEG signal analysis, ReLU allows the model to capture complex spatial and temporal relationships across multiple brain regions, which linear activations cannot represent. For example, subtle patterns such as frontal alpha asymmetry or channel-specific oscillations are effectively highlighted by ReLU, while irrelevant low-amplitude noise is suppressed.

ReLU is applied **after both convolutional and dense layers**, ensuring that only the most relevant features are propagated through the network. This non-linearity accelerates convergence during training, mitigates the vanishing gradient problem commonly seen in deep networks, and contributes to more robust and accurate depression classification. By focusing the network on significant activations, ReLU enhances feature learning and

improves the predictive performance of the CNN–LSTM model on EEG_Data_New dataset segments.

6.3.5 ADAM OPTIMIZATION

The **Adam optimizer (Adaptive Moment Estimation)** is employed to train the CNN–LSTM network efficiently on the EEG_Data_New dataset. Adam combines the advantages of **momentum** and **RMSProp**, dynamically adjusting the learning rate for each parameter based on both the first (mean) and second (variance) moments of gradients. This allows the optimizer to converge faster and more smoothly compared to traditional stochastic gradient descent, even in the presence of noisy and high-dimensional EEG data.

Adam’s adaptive nature helps prevent oscillations during training and stabilizes learning across multiple layers, including convolutional, LSTM, and fully connected layers. By maintaining per-parameter learning rates, the optimizer ensures that each network weight is updated appropriately, which is crucial for capturing the subtle spatial-temporal patterns in EEG signals related to depression. Overall, Adam contributes to faster convergence, stable training, and improved predictive accuracy for the depression classification task.

6.3.6 HUBER LOSS FUNCTION

The **Huber loss function** is used as the primary loss metric during model training to enhance robustness against outliers in EEG data. Unlike Mean Squared Error (MSE), which can be heavily influenced by extreme values, Huber loss combines both **L1 and L2 loss characteristics**, using a squared loss for small errors and absolute loss for larger deviations. This makes it particularly effective for EEG signals, which may contain occasional artifacts or abnormal spikes even after pre-processing.

By applying Huber loss, the CNN–LSTM network is trained to focus on generalizable patterns rather than being overly sensitive to extreme signal fluctuations. This improves stability, reduces the risk of overfitting, and enhances overall model reliability. In combination with the Adam optimizer, Huber loss ensures efficient, accurate, and robust training of the depression detection model. Ultimately, this loss function helps maintain a balanced learning process, especially when dealing with noisy biomedical data such as EEG recordings.

6.3.7 CLASSIFICATION AND PREDICTION MODULE

The **Classification and Prediction Module** is the final stage of the CNN–LSTM pipeline, responsible for translating learned spatial-temporal EEG features into meaningful clinical outcomes. High-level features extracted from the LSTM layers are passed to a series of **fully connected (dense) layers** with dimensions $[64 \rightarrow 32 \rightarrow 2]$. These layers progressively reduce feature dimensionality while preserving essential information required for accurate classification.

The final output layer employs the **Softmax activation function** to generate probability scores for each class, indicating the likelihood of a subject being **Depressed** or **Healthy**. During inference, the model predicts the class with the highest probability, effectively mapping complex EEG signal patterns into a clinically interpretable result.

This module integrates the spatial features learned by the CNN and the temporal dynamics captured by the LSTM, completing the end-to-end pipeline for depression detection. By combining robust feature extraction, sequential modeling, and probabilistic classification, the module ensures reliable, accurate, and interpretable predictions, making it suitable for real-world applications such as clinical diagnostics, mental health monitoring, and telehealth platforms using the EEG_Data_New dataset.

6.4 CODE

```
import numpy as np
import os
import tensorflow as tf
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

# --- Import our model builder ---
# We assume 'model.py' is in the same 'scripts/' folder
from model import build_dephnn_model, N_CHANNELS, N_TIMESTEPS

# --- Configuration ---
DATA_PATH = 'data/preprocessed/'
MODEL_SAVE_PATH = 'models/'
HISTORY_SAVE_PATH = 'models/'

# Create directories if they don't exist
if not os.path.exists(MODEL_SAVE_PATH):
```

```
os.makedirs(MODEL_SAVE_PATH)

# --- 1. Load Data ---
print("Loading preprocessed data...")
try:
    X = np.load(os.path.join(DATA_PATH, 'X_data.npy'))
    y = np.load(os.path.join(DATA_PATH, 'y_labels.npy'))
except FileNotFoundError:
    print("\n--- ERROR ---")
    print(f"Could not find X_data.npy or y_labels.npy in '{DATA_PATH}'")
    print("Please run 'python scripts/preprocess_data.py' first.")
    exit()

print(f"Data (X) loaded with shape: {X.shape}")
print(f"Labels (y) loaded with shape: {y.shape}")

# --- 2. Format Labels ---
# Our model's output layer is 'softmax' with 2 units.
# It requires labels to be "one-hot encoded".
# e.g., 0 -> [1, 0] (Healthy)
#       1 -> [0, 1] (Depressed)
y_categorical = to_categorical(y, num_classes=2)
print(f"Labels (y) one-hot encoded to shape: {y_categorical.shape}")

# --- 3. Split Data (70% Train, 20% Val, 10% Test) ---
# This is a two-step split.
# First, split into 70% Train and 30% (Val + Test)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y_categorical, test_size=0.30, random_state=42, stratify=y_categorical
)

# Second, split the 30% temp data into 2/3 (20%) and 1/3 (10%)
# This gives 0.30 * 0.666 = 0.20 (20%) for Validation
# and 0.30 * 0.333 = 0.10 (10%) for Test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.333, random_state=42, stratify=y_temp
)

print("\nData splitting complete:")
print(f" - Training samples: {len(X_train)}")
print(f" - Validation samples: {len(X_val)}")
print(f" - Test samples: {len(X_test)}")

# --- 4. Build and Compile Model ---
# Build the model we defined in 'model.py'
model = build_dephnn_model(input_shape=(N_CHANNELS, N_TIMESTEPS))
model.summary()

# Compile the model as specified in the paper
```

```
# Optimizer: ADAM
# Loss: Huber's Loss
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
loss_function = tf.keras.losses.Huber()

# --- THIS BLOCK IS NOW CORRECTED ---
model.compile(
    loss=loss_function,
    optimizer=optimizer,
    metrics=['accuracy', 'mae'] # 'mae' is Mean Absolute Error
)
# --- END OF CORRECTION ---

print("\nModel compiled successfully.")

# --- 5. Define Callbacks ---
# These help us manage the training process.

# ModelCheckpoint: Save the *best* model (lowest val_loss)
# We use the .keras format, which is the modern standard
checkpoint_path = os.path.join(MODEL_SAVE_PATH, 'DepHNN_model_best.keras')
model_checkpoint = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)

# EarlyStopping: Stop training if the model stops improving
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=15, # Stop after 15 epochs of no improvement
    restore_best_weights=True,
    verbose=1
)

# --- 6. Train the Model ---
print("\nStarting model training...")

EPOCHS = 100
BATCH_SIZE = 64 # From the paper

history = model.fit(
    X_train, y_train,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    validation_data=(X_val, y_val),
    callbacks=[model_checkpoint, early_stopping]
)
```

```
print("Model training complete.")

# --- 7. Save Final Model and History ---
# Save the final (best) trained model
final_model_path = os.path.join(MODEL_SAVE_PATH, 'DepHNN_model_final.keras')
model.save(final_model_path)
print(f"Final trained model saved to: {final_model_path}")

# Save the training history as a .npy file
history_path = os.path.join(HISTORY_SAVE_PATH, 'training_history.npy')
np.save(history_path, history.history)
print(f"Training history saved to: {history_path}")

# --- 8. Save the Test Set ---
# We save the 10% test set to be used *only* in Phase 5
print("Saving 10% test set for evaluation...")
np.save(os.path.join(DATA_PATH, 'X_test.npy'), X_test)
np.save(os.path.join(DATA_PATH, 'y_test.npy'), y_test)
print("Test set saved.")

print("\nPhase 4: Model Training is complete.")
```

CHAPTER 7

TESTING